

SlickChat — Contratos

Este documento define os contratos iniciais entre componentes do sistema SlickChat. Ele será evoluído ao longo do desenvolvimento.

1. Gateway ↔ Cliente (WebSocket)

1.1 Conexão

Endpoint:

```
GET /socket?ticket=<ws_ticket>
```

Observação:

O ticket WebSocket é emitido pela API e deve ser usado uma única vez.

1.2 Formato de Mensagens

Envelope padrão

```
{
  "type": "string",
  "payload": {}
}
```

1.3 Enviar mensagem (Client → Gateway)

```
{
  "type": "send_message",
  "payload": {
    "room_id": "uuid",
    "content": "string"
  }
}
```

1.4 Receber mensagem (Gateway → Client)

```
{
  "type": "message.received",
  "payload": {
    "message_id": "uuid",
    "room_id": "uuid",
    "sender_id": "uuid",
    "content": "string",
    "timestamp": "ISO8601"
  }
}
```

1.5 ACK de envio (Gateway → Client)

```
{
  "type": "message_ack",
  "payload": {
    "status": "received"
  }
}
```

2. Gateway ↔ Redis

2.1 Estruturas

Conexões por usuário

```
user_connections:{user_id} -> Set(connection_id)
```

Dados da conexão

```
connection:{connection_id} -> Hash
{
  user_id,
  gateway_id
}
```

2.2 Pub/Sub (entrada de mensagens)

Canal por conexão

```
connection:{connection_id}
```

2.3 Mensagem recebida via Pub/Sub

```
{
  "type": "deliver_message",
  "payload": {
    "connection_id": "uuid",
    "data": {
      "message_id": "uuid",
      "room_id": "uuid",
      "sender_id": "uuid",
      "content": "string",
      "timestamp": "ISO8601"
    }
  }
}
```

3. Gateway ↔ Worker

3.1 Contrato via eventos (Kafka)

O Gateway publica eventos em `message-events` e os workers consomem por grupo:

- `fanout-group`
- `persistence-group`

4. Worker ↔ Redis

4.1 Descoberta de conexões

```
GET user_connections:{user_id}
GET connection:{connection_id}
```

4.2 Publicação para conexão

```
PUBLISH connection:{connection_id} <event>
```

5. Identidade de Infra

5.1 Gateway

```
var gatewayID = "uuid"
```

5.2 Connection

```
type Connection struct {  
    ConnectionID string  
    UserID       string  
    GatewayID    string  
}
```

6. Padrão de Erros (WebSocket)

6.1 Estrutura

```
{  
  "type": "error",  
  "payload": {  
    "code": "string",  
    "message": "string"  
  }  
}
```

6.2 Exemplos

```
{  
  "type": "error",  
  "payload": {  
    "code": "invalid_payload",  
    "message": "content is required"  
  }  
}
```

```
{  
  "type": "error",  
  "payload": {  
    "code": "unauthorized",  
    "message": "invalid token"  
  }  
}
```

Este documento é a fonte da verdade para comunicação entre serviços. Qualquer mudança de contrato deve ser refletida aqui primeiro.